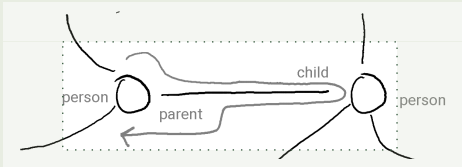


What Are Inductive Properties?



Exploring a local fragment of a model instance to check a parent-child property.

In contrast, the type of an arithmetic expression is not directly computable by just examining the instance object in question (an expression node) and a small number of its neighbors. Consider the rule for typing a binary addition expression: *The result of binary addition is an integer if both of its arguments are integers*. This rule, similarly to the other constraints we considered before, can be split into two parts:

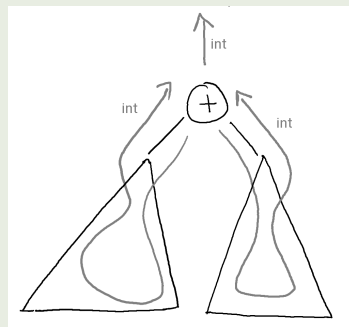
- Premises (inductive):** Both sub-expressions evaluate to an integer number
- Conclusion (structural):** The result of binary addition is an integer

To establish that the conclusion holds, we first need to establish the premises. Often, and also in this case, enforcing the premises may require exploration of an arbitrary large abstract-syntax tree, using the same rule applied to a smaller part of the model. What if the left operand is an addition expression itself? And what if the left operand of the left operand is an addition again? You can see that we might be dealing with an arbitrary large sub-tree whose type is not directly known. We shall apply the same typing rule to smaller and smaller sub-expressions, until we hit the leaves (constants and variable references), where we can decide with certainty that their type is integer, without invoking the rule recursively.

An *inductive property* is a property which requires multiple recursive checks of itself on decreasing pieces of syntax. In language implementation, we usually encounter mutually recursive inductive properties—sets of rules that recursively invoke each other. Establishing that they hold requires exploring arbitrary large parts of the instance. This resembles reflexive transitive closure. Indeed, transitive closure is an example of an inductive property.

We use *structural induction*, which differs from mathematical induction you may recall from high school. Mathematical induction derives facts for natural numbers if they hold for smaller numbers. Structural induction establishes that a property holds for a syntax tree if it holds for smaller sub-trees. The process terminates, since at every inductive step we are considering smaller trees, until we arrive at basic terms, which can be typed non-inductively (without further recursion).

In Chapter 5, we focused on requirements (restrictions) that could be expressed directly in terms of meta-model types, through a localized inspection. For example: *A person object should be included among its child object's parents*. This property is directly expressible as a computation over a fixed number of objects, without navigating arbitrarily far from the context object in the syntax graph.



Establishing an inductive property may require exploration of arbitrary large sub-trees of an AST.